



Deep Learning Report

Group 2

Balazs Molnar — 20251004

Beatriz Gil — 20231603

Inês Chainho — 20231653

Jakub Bujak — 20251053

Katarina Dvornak — 20251052

December 16, 2025

1 Introduction

Accurate identification of rare species from images is challenging because many threatened taxa are underrepresented in datasets and difficult to distinguish even for experts. Recent work such as BioCLIP shows that deep models trained on large, taxonomically rich image collections can provide strong visual features across the tree of life, but these models still need careful adaptation to specific datasets and label spaces. This project aims to build a deep-learning system that predicts the family of an organism from its image, using a rare-species dataset derived from the Encyclopedia of Life and curated in the BioCLIP study, with nearly 12 000 images labeled at kingdom, phylum, and family levels and exhibiting strong class imbalance and variable image quality. Several CNN-based approaches are evaluated, from a custom CNN trained from scratch to transfer learning with Xception, EfficientNetV2B0, EfficientNetB0, and ConvNeXt-Small, with the report detailing data exploration, preprocessing, custom CNN tuning, transfer-learning and fine-tuning strategies, and a comparative assessment using accuracy and macro F1 as primary metrics.

2 Approach

2.1 Exploratory Data Analysis

Prior to model development, a comprehensive exploratory data analysis (EDA) was conducted to assess dataset quality and characteristics. The metadata contained 11,983 images with identifiers, taxonomic information (kingdom, phylum, family), and image paths. Data types, non-null counts, and missing values were verified, while frequency counts and visualizations of phylum and family labels highlighted class imbalance. Data integrity and content were checked by sampling images, verifying file existence, and displaying random images with their labels. Image resolution variability was characterized by sampling up to 1,000 images, extracting their width and height with Pillow, and identifying the smallest and largest pixel areas to approximate the range of image sizes.

2.2 Data Pre-Processing

The data preprocessing process ensured integrity and suitability for training the final model. Metadata was mapped to local file paths, and rows with missing file paths or family labels were removed. Integrity checks verified that all image files existed, and duplicates were eliminated to ensure unique samples. The clean dataset was then partitioned with a stratified split to preserve class distribution: 80% for training, 10% for validation, and 10% for testing.

2.3 Final Model

We experimented with several models, whose results are presented in the "Results" section, but **ConvNeXt-Small** clearly stood out. The approach is described in the following subsections:

2.3.1 Architecture and Training Strategy

ConvNeXt-Small was selected as the main model due to its strong performance on natural image classification tasks and its effectiveness in transfer learning scenarios with medium-sized datasets and many classes. The model was initialized with ImageNet pre-trained weights, and the original classification head was removed so that ConvNeXt could be used as a feature extractor.

A custom classification head was added on top of the ConvNeXt output, consisting of Global Average Pooling, Dropout (0.3), a Dense layer with 512 ReLU units, Batch Normalization, a second Dropout (0.2) layer, and a final Dense layer with 202 Softmax units, corresponding to the number of families in the dataset.

Images were resized to 224×224 pixels and loaded using Keras' image data generators. Data augmentation was applied only to the training set and included random rotations, translations, zoom, shearing, horizontal flips, and brightness variations. Validation and test sets were processed only with ConvNeXt's preprocessing function. Due to strong class imbalance, class weights were computed from the training labels and applied during optimization.

2.3.2 Training Procedure

Training was performed in three stages using a fine-tuning strategy. In the first stage, the ConvNeXt backbone was completely frozen and only the custom classification head was trained for 8 epochs using the Adam optimizer with a learning rate of 3×10^{-4} . Early stopping, learning-rate reduction, and model checkpointing were used to prevent overfitting and retain the best model.

In the second stage, the top 30% of the ConvNeXt layers were unfrozen to allow higher-level features to adapt to the dataset. Training continued for up to 15 epochs with a reduced learning rate of 1×10^{-4} , while the remaining layers stayed frozen to preserve general visual representations.

In the third stage, the full backbone was fine-tuned. Batch normalization layers were kept frozen for stability, and mixed-precision training was enabled to reduce GPU memory usage. Due to hardware constraints, the batch size was reduced, and training was performed with a learning rate of 1×10^{-5} for up to 20 epochs. The same callback strategy was maintained across all stages.

2.3.3 Denoising Autoencoder

In the final stages of our project, we decided to build a basic denoising convolutional autoencoder as an attempt to improve even more our model accuracy. We isolated the train and val images in two different `tf.data.Dataset` objects, and normalized and added noise to both using batch-wise tensor operations executed by TensorFlow, which fastened the process. After, we built a basic encoder, adapting the input size.

3 Results of All Attempted Models

3.1 Custom CNN

The automated hyperparameter optimization for the scratch-trained baseline, run in two phases, showed that learning rare-species features from random initialization was not feasible under the available data and compute constraints. In the shallow two-block search, 30 trials only reached about 1.7% validation accuracy, close to random guessing for 202 classes, while the deeper Dynamic Depth search, with 3–5 convolutional blocks and larger heads, improved validation accuracy only to roughly 2% before being limited by high training times and early stopping. Overall, even systematically tuned custom CNNs failed to learn effective representations, which motivated abandoning scratch training and focusing exclusively on transfer learning models.

3.2 EfficientNetV2B0

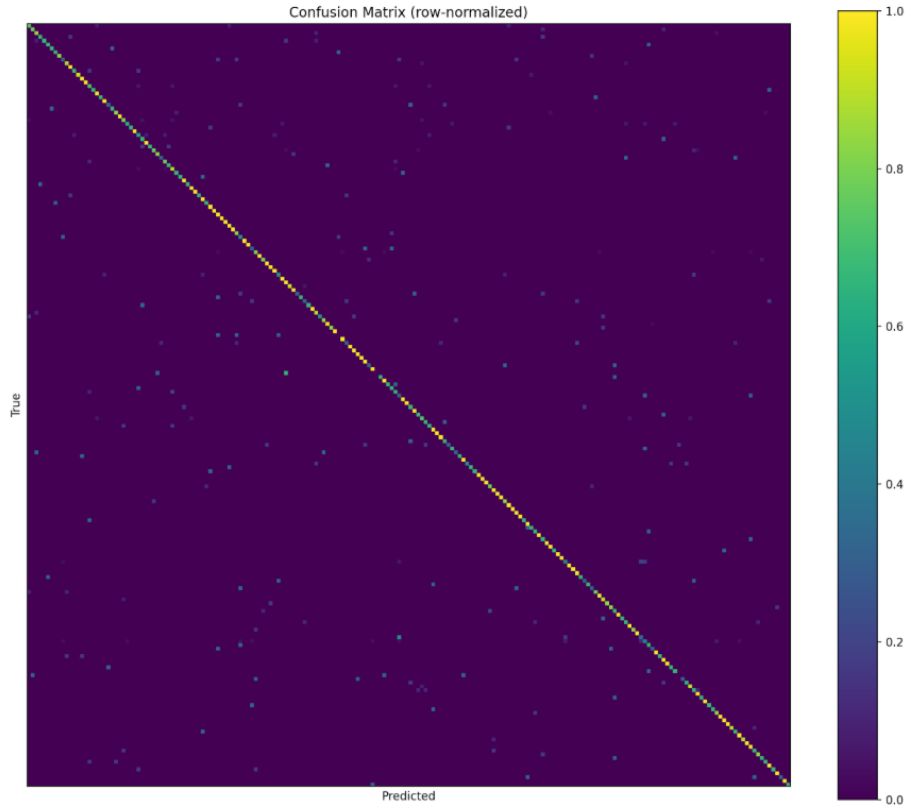
During fine-tuning, training accuracy remains in the low- to mid-70% range, while validation accuracy stabilizes in the high-60% range and validation loss decreases slightly, indicating useful learning without strong overfitting. Regularization (BatchNormalization, Dropout, data augmentation, class weights) keeps the training-validation gap moderate, showing good generalization. On the independent test set, the model achieves 71.89% accuracy and a macro F1-score of 69.81%, demonstrating balanced performance across classes. The classification report and confusion matrix show rare families remain challenging, but transfer learning provides meaningful recall for many underrepresented species, making this an effective approach.

3.3 EfficientNetB0

The EfficientNetB0 model outperformed previous custom CNNs even on the raw dataset, achieving a macro F1-score of 71.05% and accuracy of 71.98%, demonstrating the benefit of using a pre-trained model. After removing confusing and low-quality images, training became more stable, and the model improved especially on rare species. The cleaned model reached 74.06% accuracy and a macro F1-score of 72.65% on the final test, showing better performance on both common and uncommon categories.

3.4 ConvNeXt-Small

The final ConvNeXt-Small model achieved 77.91% accuracy and 76.64% macro F1 on the test set, with macro precision and recall of 78.38% and 78.71%, showing balanced performance across frequent and rare classes. Compared to earlier CNNs and smaller pre-trained models, ConvNeXt-Small converged faster, was more stable, and generalized better. Remaining errors mainly involved visually similar families or very rare classes. Overall, staged fine-tuning and strong regularization make it highly effective for large-scale biological image classification.



Results of top 15 confusions (True → Pred: count):

- plethodontidae → rhyacotritonidae: 6
- plethodontidae → salamandridae: 5
- carcharhinidae → sphyrnidae: 3
- rhyacotritonidae → hynobiidae: 3
- salamandridae → rhyacotritonidae: 3
- acroporidae → euphylliidae: 2
- agariciidae → dendrophylliidae: 2
- balistidae → labridae: 2
- bovidae → cervidae: 2
- chamaeleonidae → agamidae: 2
- cryptobranchidae → ambystomatidae: 2
- dasyatidae → gymnuridae: 2
- delphinidae → sphyrnidae: 2
- diplodactylidae → lacertidae: 2
- geoemydidae → podocnemididae: 2

The top 15 misclassifications for the ConvNeXt-Small model mostly occur between phylogenetically or morphologically related families, such as salamander families with similar shapes and overlapping traits. Other errors appear between requiem vs. hammerhead sharks, antelope-like vs. deer-like ungulates, and between related lizard and turtle families, reflecting the difficulty of separating fine-grained taxa when images are downscaled

and some diagnostic details are lost.

4 Discussion

The experiments show a progression from ineffective scratch-trained CNNs to strong transfer learning models, highlighting the difficulty of rare-species classification and the value of pre-trained backbones. Custom CNNs reached only 1.7–2% validation accuracy for 202 classes, showing that learning from random initialization is infeasible with the available data and compute.

Transfer learning with EfficientNetV2B0 and EfficientNetB0 improved test accuracy to around 72% and macro F1 near 70%, demonstrating that pre-trained features with augmentation and class weighting generalize well. Cleaning a small number of high-loss, likely mislabeled images further increased accuracy to 74.06% and macro F1 to 72.65%, especially benefiting rare classes.

ConvNeXt-Small achieved the best results, with 77.91% accuracy and 76.64% macro F1, showing balanced performance and good generalization. Training was stable with staged fine-tuning, while remaining errors mainly involved visually similar or very rare classes. Overall, transfer learning with modern CNNs is effective for large-scale, imbalanced biological image classification, though still limited by data scarcity and label noise.

5 Conclusion

This project demonstrates that transfer learning with modern convolutional architectures is an effective strategy for large-scale rare-species classification with strong class imbalance. Starting from custom CNNs that remained near chance level, progressively stronger pre-trained backbones and staged fine-tuning led to substantial improvements, with ConvNeXt-Small achieving 77.91% accuracy and a macro F1-score of 76.64% on the test set, and EfficientNet-based models also reaching macro F1-scores above 70%. Systematic preprocessing, including integrity checks, stratified splitting, data augmentation, class weighting, and targeted removal of mislabeled or low-quality images, was essential to stabilize training and obtain balanced performance across both frequent and rare families.

Despite these results, performance is still limited by label noise, strong class imbalance, and the need to downscale very high-resolution images, which may remove fine-grained morphological cues. As part of future work, we would have liked to train the final classification model using the previously developed convolutional denoising autoencoder as a preprocessing stage, in order to reduce background clutter and sensor noise before classification. To further address imbalance and rare classes, alternative loss functions such as focal loss and its variants could be explored. Finally, leveraging the taxonomic hierarchy (kingdom–phylum–family) through hierarchical classification frameworks may improve consistency and robustness for closely related species.

6 Bibliography

- [1] <https://www.pnas.org/doi/10.1073/pnas.0405785101>
- [2] <https://www.geeksforgeeks.org/deep-learning/fine-grained-image-classification>
- [3] <https://www.geeksforgeeks.org/machine-learning/auto-encoders/>
- [4] <https://www.geeksforgeeks.org/machine-learning/denoising>
- [5] <https://www.tensorflow.org/tutorials/generative/autoencoder>
- [6] <https://arxiv.org/abs/1603.06560>
- [7] <https://arxiv.org/abs/1603.06560>
- [8] <https://www.geeksforgeeks.org/deep-learning/transfer-learning-fine-tuning>
- [9] <https://www.identifyshell.org/blog-fine-tuning-efficientnetv2-models.php>
- [10] <https://neptune.ai/blog/transfer-learning-guide-examples-for-images-and-text>
- [11] <https://arxiv.org/abs/1905.11946>
- [12] <https://keras.io/api/applications/efficientnet/>
- [13] <https://arxiv.org/abs/1911.00068>
- [14] <https://www.jmlr.org/papers/volume11/vincent10a/vincent10a.pdf>
- [15] <https://arxiv.org/abs/2207.11771>
- [16] <https://arxiv.org/abs/1708.02002>
- [17] <https://www.sciencedirect.com/science/article/pii/S2772375524003654>
- [18] <https://openreview.net/forum?id=7HEMpBTb3R>
- [19] <https://www.kaggle.com/code/ztrollk/optimizing-efficientnetv2-resnet-mobilenet>
- [20] <https://github.com/tensorflow/tensorflow>
- [21] <https://github.com/keras-team/keras>
- [22] <https://www.tensorflow.org/guide/gpu>
- [23] <https://keras.io/api/applications/convnext/>
- [24] https://keras.io/guides/transfer_learning/
- [25] <https://keras.io/api/callbacks/>